A Project Report

On

GuardShield AI: A Real-Time Sidecar Proxy for Prompt-Injection Defense and Hallucination Detection, with an Audit of Format-Triggered Over-Defense in Prompt-Injection Detectors

Submitted to

Amity University Uttar Pradesh

in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

Shourya Solanki (A2305223569)

Rachit Ryan Chug (A2305223166)

Dhruv Raj Singh (A2305223191)

under the guidance of

Dr. Abhishek Kaushal

AMITY SCHOOL OF ENGINEERING AND TECHNOLOGY (ASET)

AMITY UNIVERSITY UTTAR PRADESH

NOIDA (U.P.)

October 2026

DECLARATION

We, Shourya Solanki, Rachit Ryan Chug and Dhruv Raj Singh, students of B.Tech (Computer Science and Engineering (Artificial Intelligence and Machine Learning)), hereby declare that the project titled "GuardShield AI: A Real-Time Sidecar Proxy for Prompt-Injection Defense and Hallucination Detection, with an Audit of Format-Triggered Over-Defense in Prompt-Injection Detectors", which is submitted by us to Amity School of Engineering and Technology (ASET), Amity University Uttar Pradesh, Noida, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning), has not previously formed the basis for the award of any degree, diploma or other similar title or recognition.

Noida

Date: _____

Shourya Solanki (A2305223569) Signature: _____

Rachit Ryan Chug (A2305223166) Signature: _____

Dhruv Raj Singh (A2305223191) Signature: _____

CERTIFICATE

On the basis of the declaration submitted by Shourya Solanki, Rachit Ryan Chug and Dhruv Raj Singh, students of B.Tech (Computer Science and Engineering (Artificial Intelligence and Machine Learning)), I hereby certify that the project titled "GuardShield AI: A Real-Time Sidecar Proxy for Prompt-Injection Defense and Hallucination Detection, with an Audit of Format-Triggered Over-Defense in Prompt-Injection Detectors", which is submitted to Amity School of Engineering and Technology (ASET), Amity University Uttar Pradesh, Noida, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning), is an original contribution with existing knowledge and a faithful record of work carried out by them under my guidance and supervision.

To the best of my knowledge this work has not been submitted in part or full for any degree or diploma to this University or elsewhere.

Noida

Date: _____

Dr. Abhishek Kaushal
(Guide)
Amity Centre for Artificial Intelligence (ACAI)
Amity University Uttar Pradesh, Noida

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. Abhishek Kaushal, Amity Centre for Artificial Intelligence (ACAI), for his continuous guidance, critical feedback and encouragement throughout this project. His suggestion to aim for a research publication shaped the second half of the work.

We thank the Head of the Department and the NTCC committee for the opportunity to carry out this project, and the faculty of the department for their support during the progress reviews.

We acknowledge the open-source community whose models, datasets and tools made this work possible, in particular the maintainers of Hugging Face Transformers, PyTorch and the public benchmarks cited in this report, and Kaggle for free GPU time.

[To be reviewed by the team: Generative AI tools (Anthropic's Claude) were used to assist with code review, experiment scripting and editing of drafts. All experiments were run, checked and interpreted by the team, and every reported number is reproduced from the committed result files. Keep, adjust or remove this sentence according to the University's policy on AI assistance.]

Shourya Solanki
Rachit Ryan Chug
Dhruv Raj Singh

TABLE OF CONTENTS

DECLARATION ii

CERTIFICATE iii

ACKNOWLEDGEMENT iv

ABSTRACT x

CHAPTER 1 INTRODUCTION 1

1.1 Purpose of the Project 1

1.2 Background and Available Alternatives 1

1.3 Problem Statement 2

1.4 Project Goals and Objectives 2

1.5 Organization of the Report 3

CHAPTER 2 SCOPE AND CONSTRAINTS 4

2.1 Scope Definition 4

2.2 Projected Budget 4

2.3 Project Constraints 5

CHAPTER 3 REQUIREMENT ANALYSIS 6

3.1 Stakeholders and Usage 6

3.2 Functional Requirements 6

3.3 Non-Functional Requirements 7

3.4 Hardware and Software Requirements 8

CHAPTER 4 LITERATURE REVIEW 9

4.1 Prompt Injection and Jailbreaks 9

4.2 Detectors and Guard Models 9

4.3 Evaluation of Detectors 9

4.4 Hallucination Detection and Fact Verification 10

4.5 Remedies for Obfuscation 10

4.6 Research Gap 11

CHAPTER 5 SYSTEM DESIGN AND METHODOLOGY 12
5.1 System Architecture 12
5.2 Stage 1: Prompt-Injection Classifier 12
5.2.1 Data 12
5.2.2 Model and Training 13
5.2.3 Inference 13
5.3 Stage 2: Fact Verifier 13
5.4 Streaming Entropy Monitor and Proxy 13
5.5 Research Methodology: Paired Perturbation Audit (Study 1) 14
5.6 Research Methodology: Mitigation Study (Study 2) 14
CHAPTER 6 RESULTS AND ANALYSIS 16
6.1 Stage 1 Detection Performance 16
6.2 Comparison with Baselines 17
6.3 Stage 2 Fact Verification 18
6.4 Latency and Multi-Turn Behaviour 19
6.5 Study 1: Failure Map of 19 Detectors 20
6.6 Why Detectors Fail: the Tokenizer 21
6.7 Study 2: Canonicalization versus Augmentation 21
6.8 Training-Data Contamination 23
CHAPTER 7 PROJECT MANAGEMENT APPROACH 24
7.1 Project Timeline 24
7.2 Project Roles and Responsibilities 25
CHAPTER 8 RISK ASSESSMENT 26
CHAPTER 9 LIMITATIONS AND FUTURE SCOPE 27
9.1 Limitations 27
9.2 Future Scope 27
CHAPTER 10 CONCLUSION 28

CHAPTER 11 INDIVIDUAL CONTRIBUTION.....29
Shourya Solanki (A2305223569) 29
Rachit Ryan Chug (A2305223166) 29
Dhruv Raj Singh (A2305223191).....29
REFERENCES 30
APPENDIX A. RESEARCH PAPER.....33
APPENDIX B. SECOND FAILURE MAP (PIDS-BENCH) 34
APPENDIX C. REPRODUCING THE RESULTS 35
APPENDIX D. WEEKLY PROGRESS REPORTS 36

LIST OF TABLES

Table 1.1: Alternatives for securing an LLM application....2

Table 2.1: Projected and actual budget....4

Table 3.1: Functional requirements....6

Table 3.2: Non-functional requirements....7

Table 4.1: Closest related work and the gap addressed....11

Table 6.1: Stage 1 results (decontaminated model, weights cd1e81a29c1eb942)....16

Table 6.2: Stage 1 compared with baselines on identical splits....17

Table 6.3: Stage 2 results on public fact-verification benchmarks....18

Table 6.4: False alarms on 20 benign multi-turn conversations (100 turns)....19

Table 6.5: Signal status of character-family cells by what the detector's tokenizer does (Gandalf × Dolly)....21

Table 6.6: Study 2 on PIDS-Bench, means over five seeds; FPR at the threshold that catches 90% of clean attacks....22

Table 7.1: Milestones and deliverables....24

Table 7.2: Roles and responsibilities (as recorded in the weekly progress reports)....25

Table 8.1: Project risk assessment....26

LIST OF FIGURES

Figure 5.1: GuardShield AI request path: each stage can stop the request or the answer 12

Figure 6.1: Confusion matrix, Stage 1 in-distribution test..... 16

Figure 6.2: Confusion matrix, Stage 1 zero-shot out-of-distribution test..... 17

Figure 6.3: Confusion matrix, Stage 2 on FEVER and HaluEval 18

Figure 6.4: Failure mode (fill) and within-condition AUROC (number) for every detector and perturbation family; hatching marks a destroyed signal. Gandalf × Dolly, 500 pairs..... 20

Figure 6.5: Benign-twin augmentation over training mixtures: recall and false-positive rate fall together..... 22

Figure 7.1: Project schedule over the 12-week NTCC period..... 24

Figure B.1: Study 1 repeated on 200 PIDS-Bench pairs; encoding as in the main failure map 34

ABSTRACT

Applications built on large language models (LLMs) face two failures that ordinary software does not: prompt injection, where text supplied by a user or a retrieved document overrides the developer's instructions, and hallucination, where the model states unsupported facts with confidence. This project builds GuardShield AI, a sidecar proxy that sits in front of an OpenAI-compatible LLM endpoint and checks both the incoming prompt and the outgoing answer in real time.

Stage 1 is a fine-tuned DeBERTa-v3-small classifier with sliding-window chunking. On a held-out test set of 698 prompts it reaches 99.00% accuracy with a 0.58% false-positive rate, and on 500 zero-shot out-of-distribution prompts 98.20% accuracy, at a median latency of 26.5 ms. Stage 2 is a cross-encoder natural-language-inference verifier that checks a generated claim against retrieved evidence; on 800 FEVER and HaluEval pairs it reaches 73.00% accuracy (AUROC 0.813). A streaming entropy monitor can halt generation when token uncertainty spikes, and HMAC-signed conversation history lowers false alarms in multi-turn chats. The full pipeline adds a median of 41.5 ms per request on a 4 GB laptop GPU.

While evaluating the classifier, the team found that robustness tests for such detectors only perturb attacks. The research part of the project therefore audits 19 open detectors with a paired protocol in which every attack and a length-matched benign twin receive the identical obfuscation. The detectors lose the ability to separate attacks from benign text in 114 of 209 detector–perturbation cells, and 45% of the cells that look robust on attacks alone keep their recall only by also blocking the benign twin. The cause is traced to each detector's tokenizer. Script-aware input canonicalization restores much of the lost signal (AUROC 0.55 to 0.84), whereas retraining on perturbed data reaches 0.62 on a new data source. The study is written up as a research paper in IEEE conference format.

Keywords: prompt injection, LLM guardrails, hallucination detection, natural language inference, robustness evaluation, Unicode canonicalization.

CHAPTER 1
INTRODUCTION

1.1 Purpose of the Project

Large language models are now used as the reasoning core of chat assistants, coding tools, customer-support agents and retrieval-augmented question answering. Their input is natural language, which means that the boundary between data and instructions is blurred: any text the model reads can try to change what it does. Their output is also natural language, produced by sampling, which means a fluent answer is not necessarily a correct one.

The purpose of this project is to give developers a practical, open and measurable safety layer for such systems. GuardShield AI is designed as a *sidecar*: a separate service that the application calls instead of the LLM, which forwards safe requests to the model and checks its answers, without changing the application or the model. The project also asks a research question that arose during its evaluation: are the robustness numbers reported for prompt-injection detectors, including our own, actually measuring robustness?

1.2 Background and Available Alternatives

Prompt injection was first described for instruction-following models as “goal hijacking” and prompt leaking (Perez and Ribeiro, 2022). Indirect prompt injection extends the threat to text the model retrieves rather than text the user types, such as web pages, e-mails or documents (Greshake et al., 2023). Attackers also hide instructions with character tricks and encodings, for example zero-width characters, homoglyphs, leetspeak or Base64 (Hackett et al., 2025).

Hallucination detection has followed two routes. Sampling-based methods such as SelfCheckGPT generate several answers and treat disagreement as a warning sign (Manakul et al., 2023), which multiplies inference cost. Uncertainty-based methods inspect the model's own probabilities, for example semantic entropy (Kuhn et al., 2023). Evidence-based methods compare the answer with retrieved sources, which is the setting of fact verification benchmarks such as FEVER (Thorne et al., 2018) and HaluEval (Li et al., 2023).

The available alternatives for a developer fall into three groups, compared in Table 1.1.

Table 1.1: Alternatives for securing an LLM application

Alternative	Examples	Strength	Limitation
Large LLM judges	Llama Guard 3/4, Qwen3Guard, WildGuard	Broad harm policy	1–12 B parameters; high latency; trained for harmfulness, not injection
Small encoder classifiers	ProtectAI DeBERTa v2, Prompt Guard 2, deepset	Fast, cheap	Brittle under obfuscation; over-blocking
Commercial APIs	Lakera Guard, Azure Prompt Shields	Managed service	Closed, paid, data leaves the premises, not reproducible
GuardShield AI (this project)	DeBERTa-v3-small + NLI + entropy	Open, ~26 ms, input and output checks	Must be measured honestly against the same failure modes

1.3 Problem Statement

Build an open, low-latency proxy that (i) blocks prompt injections and jailbreaks before they reach the LLM, (ii) flags generated claims that contradict the available evidence, and (iii) does so with measured, reproducible accuracy, false-positive rate and latency on public benchmarks. In addition, determine whether standard robustness evaluations of prompt-injection detectors can tell a robust detector from one that simply blocks every unusual-looking input, and which remedy restores the detector's discrimination.

1.4 Project Goals and Objectives

- Fine-tune a small transformer classifier that detects prompt injections with high recall and a low false-positive rate, in under 50 ms on a laptop GPU.
- Build a fact-grounding verifier that checks generated claims against retrieved evidence and separates contradiction from mere lack of evidence.
- Integrate both stages, a streaming entropy monitor and multi-turn conversation handling into an OpenAI-compatible FastAPI proxy.
- Evaluate every component on public benchmarks with confidence intervals, per-sample logs and data provenance (dataset and weight hashes).
- Audit 19 open detectors with a paired benign-twin protocol, explain their failures, and compare canonicalization with retraining as remedies.

- Publish the research outcome as a paper, as required by the NTCC guidelines.

1.5 Organization of the Report

Chapter 2 defines the scope, budget and constraints, and Chapter 3 the requirements. Chapter 4 reviews the literature. Chapter 5 describes the system design and the research methodology, and Chapter 6 presents the results. Chapters 7 and 8 cover project management and risk. Chapter 9 discusses limitations and future scope, Chapter 10 concludes, and Chapter 11 records individual contributions. References and appendices follow.

CHAPTER 2
SCOPE AND CONSTRAINTS

2.1 Scope Definition

The project covers the design, implementation and evaluation of GuardShield AI and the research audit that grew out of it. In scope:

- Stage 1 prompt-injection classifier: data curation from public corpora, training, decontamination and evaluation.
- Stage 2 fact verifier based on a pretrained NLI cross-encoder, with sentence-aligned evidence windows.
- Streaming entropy monitor for token-level uncertainty during generation.
- FastAPI sidecar proxy with an OpenAI-compatible chat endpoint, multi-turn scanning and HMAC-signed assistant history.
- Evaluation harness, comparison with ProtectAI DeBERTa-v3 and a TF-IDF baseline, and latency measurement.
- Research audit (Study 1) of 19 detectors on two attack sources, with a contamination audit and a tokenizer-level explanation.
- Mitigation study (Study 2): canonicalization versus benign-twin augmentation, five training seeds per arm.

Out of scope: training a new LLM; defending against attackers who adapt to GuardShield specifically; commercial APIs (excluded for cost, terms of service and reproducibility); and the Hindi / Hinglish benchmark, which was moved to the eighth-semester project because writing native-speaker text in three scripts for 400 prompts was too large for this semester.

2.2 Projected Budget

The project was planned and completed with no expenditure, as summarised in Table 2.1.

Table 2.1: Projected and actual budget

Item	Source	Cost (INR)
Laptop with NVIDIA RTX 3050 (4 GB)	Team member's own machine	0
Cloud GPU (2 × NVIDIA T4, about 30 GPU-hours per week)	Kaggle free tier	0

Item	Source	Cost (INR)
Models (DeBERTa-v3, NLI cross-encoder, 19 audited detectors)	Hugging Face Hub, open licences	0
Datasets (SPML, deepset, Gandalf, Dolly, PIDS-Bench, FEVER, HaluEval, Aya)	Public research releases	0
Software (Python, PyTorch, Transformers, FastAPI, Overleaf)	Open source / free tier	0
Total		**0**

2.3 Project Constraints

- **Hardware:** a 4 GB laptop GPU limits model size; the 8–12 B judges were scored on Kaggle, and retraining used the small DeBERTa backbone only.
- **Time:** 12 weeks within the NTCC calendar (commencement 20 July 2026, end date 9 October 2026, final report 26 October 2026).
- **Licences and gating:** several detectors (Llama Guard, Prompt Guard, Sentinel) require licence acceptance on Hugging Face; one (PIGuard) executes remote code and was not enabled.
- **Data:** only public data with known licences; no personal data; PIDS-Bench is restricted to research use.
- **Reproducibility:** every reported number had to be regenerated from committed files, with pinned model revisions and dataset hashes.
- **Plagiarism:** the report must stay below a 15% similarity index (Turnitin).

CHAPTER 3
REQUIREMENT ANALYSIS

The requirements were derived from the problem statement in Section 1.3 and from the threat model of Section 1.2: an attacker controls some of the text that reaches the LLM, and the LLM may produce unsupported claims. They are grouped into functional requirements, which state what GuardShield must do, and non-functional requirements, which state how well it must do it. Each requirement was given a test or a benchmark so that it could be checked rather than assumed.

3.1 Stakeholders and Usage

Three groups use the system. Application developers place the proxy between their application and the LLM and need it to be a drop-in replacement for the model's API. Security and compliance teams need clear block decisions, a low false-alarm rate and logs that show why a request was stopped. End users never see the proxy directly, but they are affected by every false alarm, because a blocked legitimate request is a failed request. The research audit adds a fourth group, researchers who evaluate detectors, who need a harness that scores any detector in the same way.

3.2 Functional Requirements

Table 3.1 lists the nine functional requirements in the order a request meets them. FR1 to FR5 cover the input side: the proxy must speak the same API as the model, score every user message, see past padding in long prompts, handle conversations rather than single prompts, and reject payloads too large to scan. FR6 and FR7 cover the output side, during and after generation. FR8 and FR9 serve the evaluation and the research study.

Table 3.1: Functional requirements

ID	Requirement
FR1	Accept OpenAI-compatible /v1/chat/completions requests and forward safe ones to a configurable upstream LLM.
FR2	Score every user message for prompt injection and block requests with risk at or above 0.5 (HTTP 403).
FR3	Scan long prompts with sliding windows so that padding cannot hide an attack beyond the first window.
FR4	Scan multi-turn conversations, using short previous turns as context, and skip assistant turns signed by the proxy.
FR5	Reject oversized payloads (HTTP 413) before inference.

ID	Requirement
FR6	Monitor token entropy during streaming and stop generation after consecutive uncertainty spikes.
FR7	Verify a generated claim against evidence: entailed, contradicted (flag) or neutral (unverified).
FR8	Provide an audit harness that builds paired datasets, scores any configured detector, computes metrics and writes reports.
FR9	Record provenance for every result: dataset SHA-256, weights SHA-256, model revision, git commit, device.

FR4 deserves a note. A short follow-up such as “now do the same for the other one” is harmless alone but can complete an attack started in the previous turn, so each user turn is scored both alone and with the turn before it. Assistant turns are the opposite case: they were produced by the model, so re-scanning them only adds false alarms, but an attacker could also paste a fake “assistant” turn into the history. Signing every relayed assistant turn with a secret key lets the proxy skip only the turns it actually relayed. The proxy requirements are checked by automated tests with a stub classifier; FR7 is checked on the FEVER and HaluEval benchmarks, and FR8 by the tests of the audit harness.

3.3 Non-Functional Requirements

Non-functional requirements turn “fast”, “accurate” and “reproducible” into targets that can be measured. Table 3.2 lists each target next to the value achieved in the final evaluation of Chapter 6.

Table 3.2: Non-functional requirements

Category	Requirement	Achieved
Latency	Stage 1 median under 50 ms on a laptop GPU	26.5 ms
Latency	Full pipeline median under 100 ms	41.5 ms
Accuracy	Stage 1 false-positive rate under 2% on held-out benign prompts	0.58%
Resources	Run inference on a 4 GB GPU or CPU	Yes
Security	Signed history with a secret key; no secret in code	HMAC-SHA256, key from environment
Reproducibility	Results regenerate from committed files	Pinned revisions, hashes, scripts
Maintainability	Automated tests in continuous integration	40 tests
Licensing	Open-source release	Apache-2.0

The latency targets follow from the purpose of a sidecar. An LLM response usually takes hundreds of milliseconds to several seconds, so a safety layer that adds tens of milliseconds is acceptable, while one that adds seconds, like an 8-billion-parameter judge, is not. The false-positive target was met on held-out clean prompts

but not under obfuscation, as Section 6.5 shows; that gap between the stated requirement and behaviour on unfamiliar formats is what led to the research study. Reproducibility is treated as a requirement rather than a convenience because several results in this field are known to be inflated by overlap between training and test data (Liu, 2026).

3.4 Hardware and Software Requirements

The system was designed for the hardware a student team or a small company can be expected to have, and uses only open-source software:

- Hardware: x86-64 machine, 8 GB RAM or more; NVIDIA GPU with 4 GB memory recommended (CPU works at lower speed).
- Software: Python 3.10+, PyTorch 2.6, Hugging Face Transformers 4.57, FastAPI, scikit-learn, NumPy and pandas.
- Cloud (research audit only): Kaggle notebook with 2 × T4 GPUs for the 8–12 B parameter judges.

All GuardShield components, including training, run on the 4 GB laptop GPU. Only the research audit needed more memory, for the generative judges with 8 to 12 billion parameters, and it used free cloud GPUs instead.

CHAPTER 4
LITERATURE REVIEW

The review is organised by theme. It covers the threat, the detectors deployed against it, how those detectors are evaluated, hallucination detection, and the remedies proposed for obfuscation. The research gap is stated at the end.

4.1 Prompt Injection and Jailbreaks

Perez and Ribeiro (2022) showed that simple instructions such as “ignore previous instructions” hijack an instruction-following model and leak its system prompt. Greshake et al. (2023) showed that the same attack works indirectly, through retrieved web pages and documents, which makes every LLM-integrated application that reads untrusted text a target. Public attack corpora used in this project come from this line of work: the SPML benchmark of system-prompt attacks (Sharma et al., 2024), the deepset prompt-injection dataset (deepset, 2023) and the Gandalf game, in which players tried to make a model reveal a password (Lakera AI, 2023).

4.2 Detectors and Guard Models

Two families of detectors are deployed. Fine-tuned encoder classifiers, such as ProtectAI's DeBERTa-v3 model, Meta's Prompt Guard 2 (Meta, 2025) and PIGuard (Li et al., 2025), are small and fast. Generative safety judges such as Llama Guard (Inan et al., 2023), Qwen3Guard (Zhao et al., 2025) and WildGuard (Han et al., 2024) apply a broad harm policy but are large and slow. HiveTraceGuard-Pro is a compact generative guard trained on obfuscated examples of both labels (Oblakov et al., 2026). GuardShield's Stage 1 builds on DeBERTa-v3 (He et al., 2023), the backbone of most encoder detectors.

4.3 Evaluation of Detectors

Li et al. (2025) introduced NotInject and defined over-defense as false alarms caused by trigger words in benign prompts. PIDS-Bench evaluates four detectors on in-distribution, out-of-distribution and obfuscated data, but its obfuscated set contains attacks only, so the false-positive rate under obfuscation is undefined (Shire and Kim, 2026). Liu (2026) audits 15 detectors on benign agent tool outputs, finds default-

threshold false-positive rates from 0% to 93%, and shows that one detector was trained on the benchmark used to evaluate it. Fairoze et al. (2026) report that Prompt Guard 2 regresses on encoded attacks. Hackett et al. (2025) measure evasion of six guards under character injection, again on attacks only.

Zhang et al. (2026) make the decisive methodological point for chat models: a model that refuses every encoded request looks safe if only harmful requests are encoded. They add benign requests under the same encoding and a “scaffold” control that keeps the encoding instruction but not the encoding, and they note that the same missing control exists for defences. This project carries that control over to detectors.

4.4 Hallucination Detection and Fact Verification

SelfCheckGPT detects hallucination from disagreement between sampled answers, without external evidence (Manakul et al., 2023). Semantic entropy measures uncertainty over meanings rather than tokens (Kuhn et al., 2023). When evidence is available, the task becomes fact verification: FEVER pairs claims with Wikipedia evidence labelled supported, refuted or not enough information (Thorne et al., 2018), and HaluEval provides question-answering pairs with hallucinated and correct answers (Li et al., 2023). TruthfulQA measures whether models repeat common misconceptions (Lin et al., 2022). GuardShield's Stage 2 uses an NLI cross-encoder, which is a single forward pass rather than many samples, and keeps the three-way label so that “not enough evidence” is not reported as a hallucination.

4.5 Remedies for Obfuscation

Two remedies exist. Canonicalization rewrites the input before the detector sees it: LLM Guard's InvisibleText scanner deletes invisible Unicode characters (Protect AI, 2024), and similar normalizers have been placed in front of a Polish safety classifier and at retrieval ingestion, reporting attack-side metrics only. Augmentation retrains the detector on perturbed text; HiveTraceGuard-Pro reports that raising recall on obfuscated attacks repeatedly raised false alarms on obfuscated benign text (Oblakov et al., 2026). Broken-Token uses the number of characters per token as an obfuscation signal, and an independent analysis found Unicode normalization inside Prompt Guard 2's tokenizer (Chester, 2025).

4.6 Research Gap

No prior work perturbs benign text exactly as it perturbs attacks when evaluating third-party prompt-injection detectors. Without that control, an evaluation cannot distinguish a detector that still recognises intent from one that blocks every unfamiliar format, nor say whether a failure can be fixed by moving the threshold. No prior work compares canonicalization and augmentation on the same detector with the benign false-positive rate measured. Table 4.1 summarises the closest work.

Table 4.1: Closest related work and the gap addressed

Work	What it does	What this project adds
Zhang et al. (2026)	Benign arm and scaffold control under encodings, for chat models	The same control for detectors, length-matched pairs, AUROC per condition
Oblakov et al. (2026)	Two-sided obfuscation for its own guard	19 third-party detectors; tests the guard itself
Shire and Kim (2026)	Detector benchmark, attack-only obfuscation	The benign arm; used as second data source
Liu (2026)	15-detector over-defense audit on clean text	Obfuscation; opposite failure directions
Fairoze et al. (2026)	Encoded attacks on 14 guards	Measures the benign-side cost
Li et al. (2025)	Over-defense from trigger words	Over-defense from format

CHAPTER 5
SYSTEM DESIGN AND METHODOLOGY

5.1 System Architecture

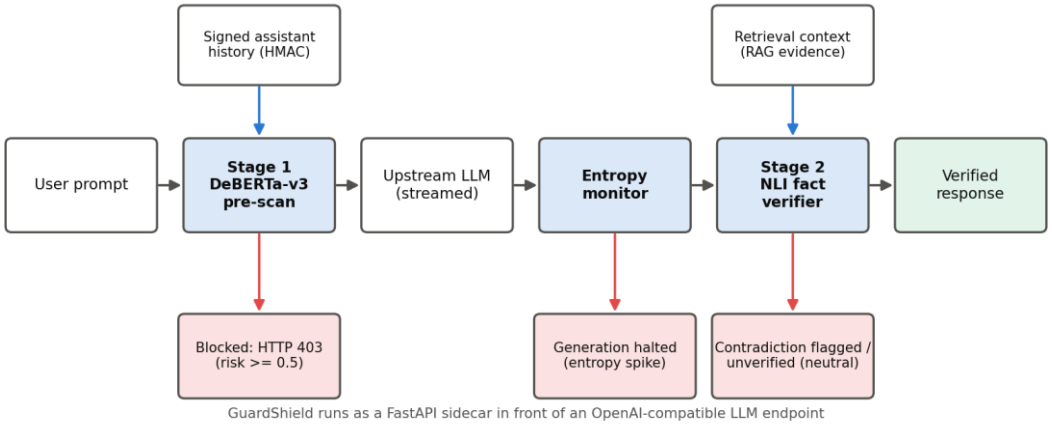


Figure 5.1: GuardShield AI request path: each stage can stop the request or the answer

Figure 5.1 shows the request path. The client sends an OpenAI-style chat request to the proxy. Stage 1 scores the prompt; a risky prompt is rejected with HTTP 403. A safe prompt is forwarded to the upstream LLM, whose streamed tokens pass through the entropy monitor. The completed answer can be checked by Stage 2 against retrieval context. The proxy signs every assistant message it relays with HMAC-SHA256, so that a later request containing the same message can be trusted without rescanning it, while a forged “assistant” turn is scanned like user text.

5.2 Stage 1: Prompt-Injection Classifier

5.2.1 Data

The training corpus combines attacks from the SPML benchmark (Sharma et al., 2024), the Jayavibhav injection suite and the deepset dataset (deepset, 2023) with benign questions from TruthfulQA (Lin et al., 2022). It is split into 3,253 training, 697 validation and 698 test prompts. A separate zero-shot set of 500 prompts contains 76 human-written jailbreaks, 174 template-shifted injections and 250 benign queries that the model never saw.

5.2.2 Model and Training

Stage 1 fine-tunes microsoft/deberta-v3-small (He et al., 2023), which has 44.3 M backbone parameters and 142.6 M parameters including its 128 k-token vocabulary. Training runs in two stages: three epochs at learning rate 2×10^{-5} with batch size 16 and maximum length 128, then one epoch at 1×10^{-5} with hard negatives (benign code, mathematics, conversational and Hinglish turns) and hard attacks. During the project audit, 29 hard-negative items were found to overlap evaluation data; they were removed and the model was retrained from scratch, and the reported numbers come from that decontaminated model.

5.2.3 Inference

Prompts of up to 126 tokens are scored in one pass. Longer prompts are split into windows of 126 tokens with a stride of 64, and the highest window score is used, so an attack cannot be hidden behind padding. A prompt is blocked when its risk score is at least 0.5.

5.3 Stage 2: Fact Verifier

Stage 2 uses the cross-encoder/nli-deberta-v3-small model, which reads an evidence passage and a claim together and outputs probabilities for entailment, neutral and contradiction. Long evidence is split into sentence-aligned windows that fit the 256-token input together with the claim; the claim is accepted if any window entails it. A claim is flagged as a hallucination when contradiction probability is at least 0.5, and reported as unverified when the evidence is neutral.

5.4 Streaming Entropy Monitor and Proxy

For each generated token the monitor computes the Shannon entropy $H = -\Sigma\, p_i \log_2 p_i$ over the top-3 normalised probabilities, whose maximum is $\log_2 3 \approx 1.585$ bits. A token above 1.25 bits is a spike, and two consecutive spikes stop generation. The proxy is a FastAPI application that enforces a 32,768-character input cap, scans each user turn both alone and with a short preceding turn as context (taking the higher risk), and strips its own signature fields before forwarding a request upstream.

5.5 Research Methodology: Paired Perturbation Audit (Study 1)

Study 1 asks whether a detector still separates attacks from benign text when both are obfuscated. Each of 500 attacks is paired with a benign prompt of the same character length, and both members of the pair receive the identical transformation under 11 families: zero-width spaces, soft hyphens, leetspeak, Cyrillic homoglyphs, Base64 and ROT13 wrappers with an instruction to decode, two scaffold controls that keep the instruction around plain text, full-width characters, combining diacritics and letter spacing. Attacks come from the Gandalf dataset (Lakera AI, 2023) and benign twins from Dolly instructions (Conover et al., 2023); a second source repeats the study on 200 pairs from PIDS-Bench (Shire and Kim, 2026).

Four measurements are taken per detector and family. The within-condition AUROC between attacks and their twins measures whether the detector still ranks attacks above benign text; its status is intact, degraded or destroyed by rules fixed before the run. Attack recall, benign false-positive rate and paired joint accuracy (attack flagged and twin passed) are read at the detector's default threshold. A cell is *fail-closed* when the benign false-positive rate rises by 20 points or more, and *fail-open* when attack recall falls by 20 points or more.

The audit covers 19 detectors: encoder classifiers (GuardShield, ProtectAI v2, deepset, fmops, Mezzo, Horizon, Sentinel, Prompt Guard v1 and 2), generative judges (Llama Guard 3 and 4, Qwen3Guard, WildGuard, HiveTraceGuard-Pro) and a perplexity baseline. Generative judges are scored without sampling, from the probability of their unsafe verdict. Every evaluated text was also checked against the published training data of the detectors that disclose it.

5.6 Research Methodology: Mitigation Study (Study 2)

Study 2 compares two remedies on GuardShield, retrained five times per arm with different random seeds: a naive canonicalizer (NFKC and deletion of all Unicode format characters), a script-aware canonicalizer (which also folds homoglyphs, collapses letter spacing and decodes Base64 and ROT13 spans, but keeps the joiners Devanagari spelling needs), decoding alone, benign-twin augmentation (a perturbed copy of each training example for six seen families) and canonicalization plus augmentation. The shipped ProtectAI v2 model receives the inference-time arms.

Each model is read at the threshold that catches 90% of its clean attacks, and families never used in training are reported separately.

CHAPTER 6
RESULTS AND ANALYSIS

6.1 Stage 1 Detection Performance

Table 6.1 reports the Stage 1 classifier on two test sets: the held-out in-distribution split, drawn from the same corpora as the training data, and a zero-shot out-of-distribution set of attack templates and benign prompts that the model never saw during training. False-positive rate (FPR) is the share of benign prompts blocked, and latency is the median time to score one prompt on the laptop GPU.

Table 6.1: Stage 1 results (decontaminated model, weights cd1e81a29c1eb942)

Split	N	Accuracy	Precision	Recall	F1	FPR	Median latency
In-distribution test	698	99.00%	99.43%	98.59%	0.9901	0.58%	26.53 ms
Zero-shot OOD	500	98.20%	99.18%	97.20%	0.9818	0.80%	26.13 ms

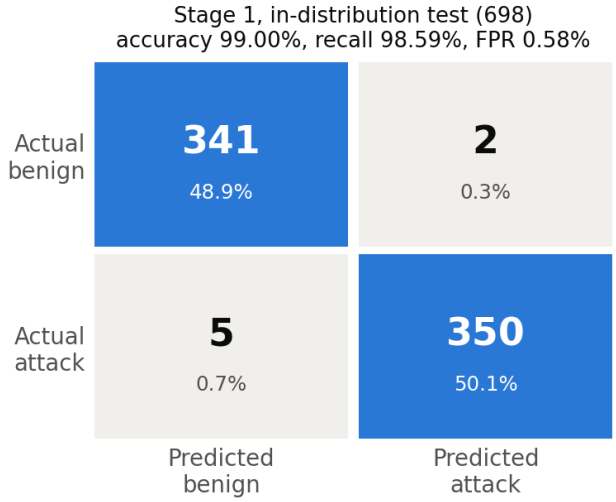


Figure 6.1: Confusion matrix, Stage 1 in-distribution test

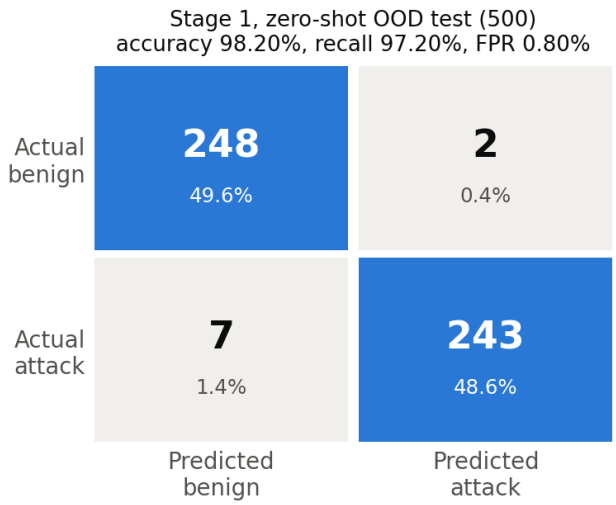


Figure 6.2: Confusion matrix, Stage 1 zero-shot out-of-distribution test

On the in-distribution test the classifier misses 5 of 355 attacks and blocks 2 of 343 benign prompts (Figure 6.1). On unseen prompts it misses 7 of 250 attacks and blocks 2 of 250 benign prompts (Figure 6.2). Sections 6.5 to 6.7 show that these clean-text numbers hide a large weakness under obfuscation, and Section 6.8 that part of the in-distribution advantage reflects overlap between SPML training text and public attack sets.

6.2 Comparison with Baselines

To judge whether these numbers are good, Table 6.2 compares GuardShield with two baselines scored on identical splits: a TF-IDF bag-of-words model with logistic regression, the simplest learned detector, and ProtectAI's DeBERTa-v3-base classifier, a widely used open-source detector.

Table 6.2: Stage 1 compared with baselines on identical splits

Model	Parameters	In-dist. accuracy	In-dist. recall	OOD accuracy	OOD recall	Latency
TF-IDF + logistic regression	linear	94.41%	91.55%	71.60%	45.60%	< 1 ms
ProtectAI DeBERTa-v3-base	184.0M	93.84%	88.73%	88.20%	78.80%	65.26 ms
GuardShield DeBERTa-v3-small	142.6M	99.00%	98.59%	98.20%	97.20%	26.53 ms

GuardShield is smaller and faster than ProtectAI's model and generalises better to the zero-shot set (97.20% against 78.80% recall). The TF-IDF baseline, which matches surface words, drops to 45.60% recall on unseen attack templates,

which shows why a contextual model is needed. ProtectAI was not trained on these splits, so the in-distribution comparison favours GuardShield.

6.3 Stage 2 Fact Verification

Stage 2 was evaluated on 400 claims from FEVER and 400 question–answer pairs from HaluEval, each half supported and half contradicted or hallucinated. Table 6.3 gives the results; recall is the share of unsupported claims that the verifier flags as contradicted.

Table 6.3: Stage 2 results on public fact-verification benchmarks

Benchmark	N	Accuracy	Recall (contradiction)	Precision	FPR	AUROC
FEVER	400	86.25%	88.00%	85.02%	15.50%	0.9496
HaluEval QA	400	59.75%	45.50%	63.64%	26.00%	0.6127
Combined	800	73.00%	66.75%	76.29%	20.75%	0.8131

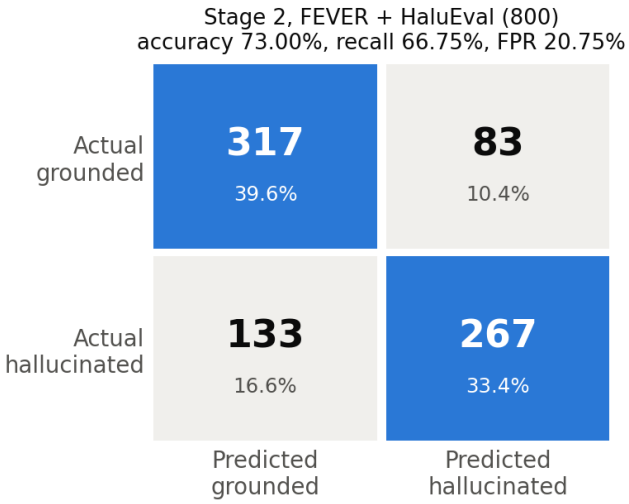


Figure 6.3: Confusion matrix, Stage 2 on FEVER and HaluEval

The verifier is strong on FEVER (AUROC 0.950), whose claims are short and checked against a single passage, and weak on HaluEval (AUROC 0.613), whose hallucinated answers are often plausible elaborations that the evidence neither confirms nor refutes (Figure 6.3). Of the 400 hallucinated items, 79 are judged neutral rather than contradicted; the system reports these as unverified instead of hiding the uncertainty.

6.4 Latency and Multi-Turn Behaviour

Over five repeats of 100 sequential requests on the laptop GPU, Stage 1 alone takes a median 20.27 ms, Stage 2 alone 20.78 ms, and the full pipeline 41.52 ms (interquartile range 11.95 ms, 95th percentile 70.70 ms), well within the 100 ms target of Chapter 3.

Multi-turn scanning was tested on 20 benign conversations of five turns each. Table 6.4 counts the false alarms of ProtectAI v2 scoring each user turn alone and of GuardShield with and without signed assistant history (FR4 in Chapter 3).

Table 6.4: False alarms on 20 benign multi-turn conversations (100 turns)

Configuration	User turns flagged (of 60)	Conversations blocked (of 20)
ProtectAI v2, each turn alone	5 (8.33%)	4 (20.00%)
GuardShield, unsigned history	5 (8.33%)	5 (25.00%)
GuardShield, signed assistant history	4 (6.67%)	3 (15.00%)

All 20 conversations are benign, so every flag is a false alarm. Signing the assistant turns that the proxy itself relayed removes them from scanning and lowers the share of blocked conversations.

6.5 Study 1: Failure Map of 19 Detectors

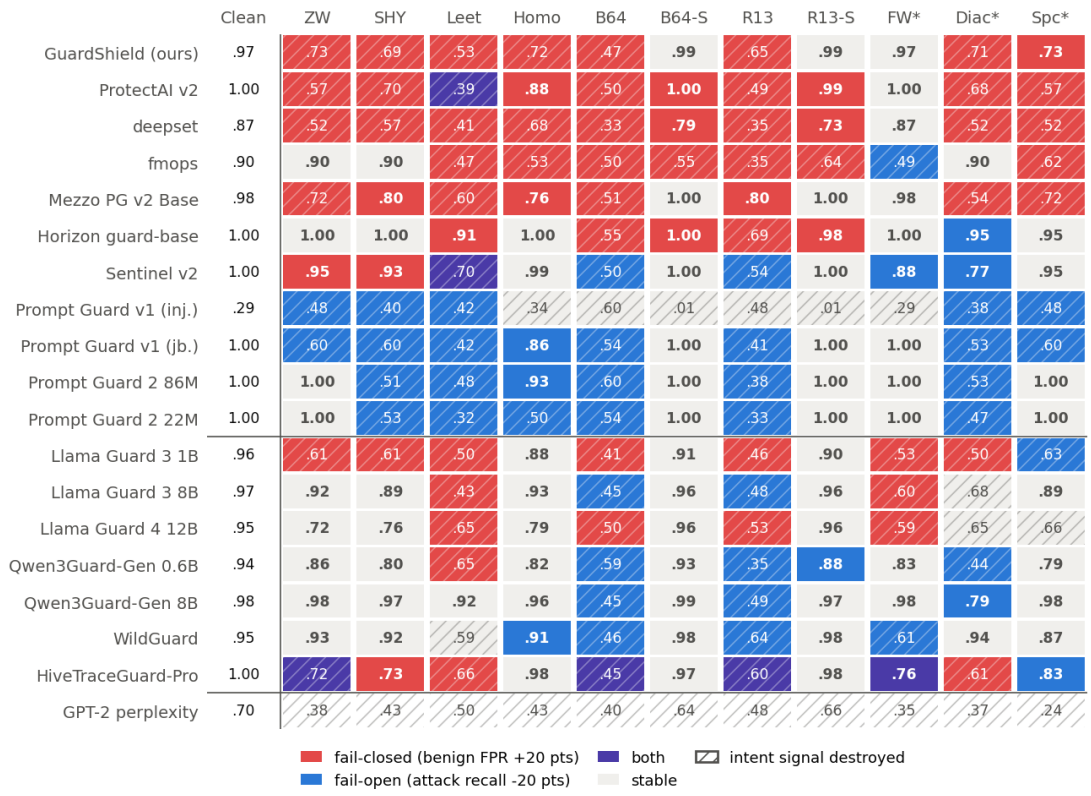


Figure 6.4: Failure mode (fill) and within-condition AUROC (number) for every detector and perturbation family; hatching marks a destroyed signal. Gandalf × Dolly, 500 pairs.

Figure 6.4 is the central result. The intent signal is destroyed in 114 of 209 detector–family cells (55%) and intact in only 55. Base64 and ROT13 wrappers destroy it in 37 of 38 cells, while the scaffold control, which keeps the decoding instruction around plain text, leaves it intact in 27 of 38; the encoding, not the wording, removes the signal.

Attack-only evaluation hides the problem. Of the 118 cells whose attack recall stays within 5 points of clean text, 53 (45%) are fail-closed: the detector keeps its recall only because it now also blocks the benign twin. For example, ProtectAI v2 catches 100.0% of Base64-wrapped attacks and blocks 100.0% of Base64-wrapped benign prompts. Conversely, Prompt Guard 2 86M falls from 99.2% recall on clean text to 0.0% under soft hyphens while its false-positive rate stays at 0.0%, a failure that a benign-only audit would score as perfect.

The direction of failure belongs to a detector–family combination, not to a detector: 7 detectors fail closed under some families and open under others.

HiveTraceGuard-Pro, the only public detector trained on obfuscated text of both labels, is near-perfect on clean text (AUROC 0.997) yet loses the signal in 5 of 11 families. Ranking detectors by perturbed-attack recall and by paired accuracy gives unrelated orders (Kendall $\tau = 0.05$), so attack-only leaderboards do not predict real robustness.

On the second source (PIDS-Bench), every one of the 105 cells with a clear failure direction keeps it, and the failure mode matches in 172 of 198 cells (Figure B.1 in Appendix B).

6.6 Why Detectors Fail: the Tokenizer

To explain the failure map, each character-family cell was classified by what the detector's own tokenizer does to the perturbed text: it either maps it back to the clean tokens, turns words into unknown tokens, or splits them into many more pieces. Table 6.5 cross-tabulates this outcome with the signal status of the cell.

Table 6.5: Signal status of character-family cells by what the detector's tokenizer does (Gandalf × Dolly)

Tokenizer outcome	Intact	Degraded	Destroyed
Perturbation normalized away	17	0	0
Collapsed into unknown tokens	0	0	4
Fragmented into more tokens	11	34	53

When the tokenizer maps the perturbed text back to the clean token sequence, the signal survives in all 17 cases; this explains why Prompt Guard 2 ignores zero-width characters, full-width forms and letter spacing. When words become unknown tokens, the signal is destroyed in all 4 cases. When the text fragments, encoder classifiers keep the signal in only 2 of 49 cells, whereas the larger generative judges sometimes read through it. Once a detector cannot read its input it returns its usual verdict: 94% of destroyed cells fail in that detector's majority direction.

6.7 Study 2: Canonicalization versus Augmentation

Study 2 asks which of two mitigations repairs the failure: canonicalizing the input before scoring, or training on obfuscated copies of both attacks and benign prompts (benign-twin augmentation). GuardShield was retrained five times per arm, and each arm was evaluated on character families and wrappers seen in training and

on held-out families. Table 6.6 gives the results on PIDS-Bench, the source not used in training.

Table 6.6: Study 2 on PIDS-Bench, means over five seeds; FPR at the threshold that catches 90% of clean attacks

Arm (GuardShield, 5 seeds)	AUROC seen char.	AUROC wrappers	AUROC held out	FPR seen char.	FPR wrappers
None (retrained)	0.55	0.46	0.62	100%	100%
Naive canonicalizer	0.77	0.46	0.62	63%	100%
Script-aware canonicalizer	0.84	0.80	0.74	46%	100%
Script-aware + decode-and-replace	0.83	0.91	0.74	46%	59%
Benign-twin augmentation	0.62	0.57	0.66	86%	94%
Canonicalize + augment	0.87	0.80	0.78	43%	92%

Script-aware canonicalization restores most of the signal on seen character families (AUROC 0.55 to 0.84) and halves the benign false-positive rate at matched recall, while augmentation reaches only 0.62. On the Gandalf pairs the two tie (0.85 against 0.86), also after removing the 158 pairs that overlap GuardShield's training data, so augmentation helps on familiar data and fails to transfer to a new source. The held-out advantage of canonicalization comes from its letter-spacing rule; diacritics, which no rule covers, defeat both arms.

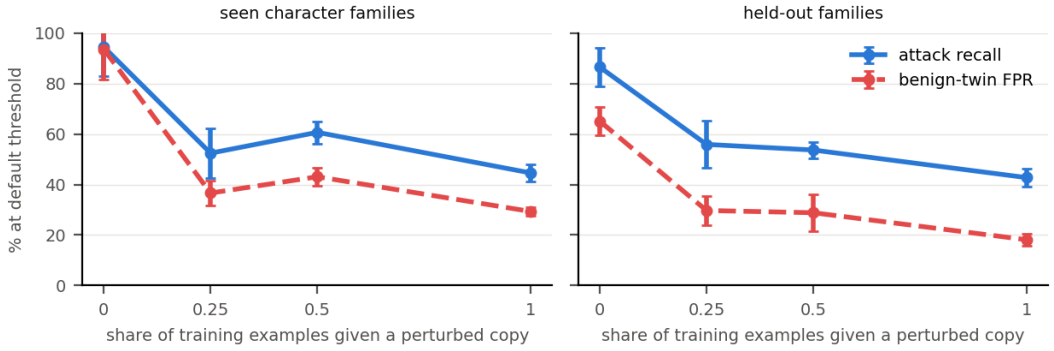


Figure 6.5: Benign-twin augmentation over training mixtures: recall and false-positive rate fall together

Decoding restores the wrapper signal but, under the rule of keeping the higher of the raw and decoded scores, leaves the benign false-positive rate at 100%; replacing the encoded input with its decoding lowers it to 59%. Figure 6.5 shows that augmentation lowers default-threshold recall from 94% to 45% together with the false-positive rate (94% to 29%), the same coupling HiveTraceGuard-Pro reported for its own model. In 1,153 native-written Hindi prompts from the Aya Dataset (Singh et al., 2024), only 1 contains a joiner that the naive canonicalizer would delete.

6.8 Training-Data Contamination

Checking the evaluation texts against the published training data of 6 detectors showed three overlaps: Horizon guard-base trained on Gandalf's training split (388 of 500 attacks), GuardShield's SPML training rows contain 157 of the Gandalf attacks, and ProtectAI v2's listed training data include most Dolly prompts. Removing each detector's contaminated pairs changes no failure mode, and no audited corpus contains any PIDS-Bench text. Overlap inflates clean-text results but does not explain the failure map.

CHAPTER 7
PROJECT MANAGEMENT APPROACH

7.1 Project Timeline

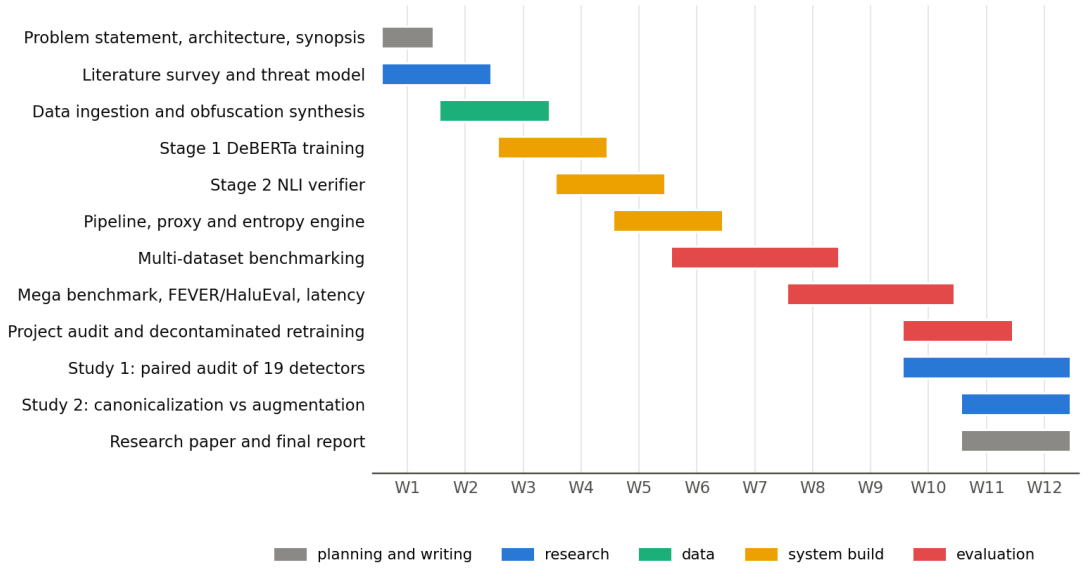


Figure 7.1: Project schedule over the 12-week NTCC period

Figure 7.1 shows the schedule followed, as recorded in the weekly progress reports. The NTCC calendar fixed the external milestones: commencement on 20 July 2026, the first progress review on 24–27 August, the second on 28 September–1 October, the end date on 9 October and the final report on 26 October 2026.

Table 7.1 lists the milestone reached in each period and the deliverable that recorded it.

Table 7.1: Milestones and deliverables

Week	Milestone	Deliverable
W1	Problem statement and architecture	WPR 1, synopsis
W2	Literature survey and threat model	WPR 2, dataset catalogue
W3–W4	Data ingestion, obfuscation synthesis, Stage 1 training	Train/validation/test splits, classifier
W5–W6	Stage 2 verifier, pipeline, proxy, entropy monitor	End-to-end proxy
W7–W8	Multi-dataset benchmarking, baselines	Comparison tables
W9–W10	Mega benchmark, FEVER/HaluEval, latency, confidence intervals	Verified results
W10–W11	Project audit, decontaminated retraining	Audited code and results
W10–W12	Study 1 audit, Study 2 mitigation, research paper	IEEE-format paper, this report

7.2 Project Roles and Responsibilities

The work was divided by role, as recorded in the weekly progress reports; Table 7.2 gives the division.

Table 7.2: Roles and responsibilities (as recorded in the weekly progress reports)

Role	Responsibilities	Participant
Lead architect	Pipeline implementation, model training, benchmarking, research audit, paper	Shourya Solanki
Research analyst	Literature survey, NLI formulation, related-work positioning	Rachit Ryan Chug
Data engineer	Multi-corpus curation, preprocessing, dataset documentation	Dhruv Raj Singh
Faculty guide	Supervision, reviews, research direction	Dr. Abhishek Kaushal

Work was coordinated through weekly meetings with the guide, a shared Git repository with automated tests, and a written specification for the research study in which every design decision and change is dated.

CHAPTER 8
RISK ASSESSMENT

Table 8.1 lists the risks identified at the start of the project and during it, with the mitigation applied.

Table 8.1: Project risk assessment

Risk	Level (L/M/H)	Likelihood	Mitigation strategy
Project size exceeds 12 weeks	H	High	Hindi benchmark moved to the 8th-semester project; scope frozen in a written specification
Laptop GPU too small for large judges	M	High	Large judges scored on free Kaggle T4 GPUs; scoring made resumable
Training/evaluation data contamination	H	Medium	Decontamination before training; contamination audit of 28 public corpora
Results not reproducible	H	Medium	Pinned model revisions, dataset and weight hashes, generated tables
Gated or unavailable models, Hub outages	M	Medium	Licences accepted in advance; automatic retry of transient errors
Competing publications on the same gap	H	High	Novelty check; narrowed claims; early write-up
Similarity index above 15%	M	Low	Report written in the team's own words; sources cited
Detector misconfiguration giving wrong scores	H	Medium	Each generative judge checked against its own generated verdicts before scoring

CHAPTER 9
LIMITATIONS AND FUTURE SCOPE

9.1 Limitations

- Stage 2 is weak on HaluEval-style answers, which are often plausible but unsupported rather than contradicted.
- Stage 1's clean-text accuracy partly reflects overlap between its SPML training data and public attack sets; under obfuscation it fails closed like other fine-tuned classifiers.
- The audit uses fixed, non-adaptive perturbations; an attacker who adapts to a specific detector would do at least as well.
- Gandalf attacks mostly target password extraction, and PIDS-Bench is dominated by templated injections, so other attack styles may behave differently.
- The canonicalizer has no rule for leetspeak or diacritics, and collapsing letter spacing cannot always recover word boundaries.
- Retraining was limited to the small DeBERTa backbone with five seeds because of the 4 GB GPU.

9.2 Future Scope

- A native-speaker Hindi, romanized Hindi and Hinglish benchmark measuring detector false positives per script (8th-semester project).
- Adding script-aware canonicalization with decode-and-replace to the GuardShield proxy as a default pre-processing step.
- Training Stage 1 on paired, two-sided obfuscated data and testing it on families held out from training.
- Improving Stage 2 with retrieval of several evidence passages and calibration on HaluEval-style answers.
- Adaptive attacks against the full pipeline, and deployment tests with real agent tool outputs.

CHAPTER 10
CONCLUSION

GuardShield AI meets its engineering goals: an open sidecar proxy that blocks prompt injections at a median of 26.5 ms with a 0.58% false-positive rate on held-out prompts, checks answers against evidence, and adds 41.5 ms per request in total on a laptop GPU, with every number reproducible from committed files.

The research part of the project shows that such numbers are not enough. When benign text is perturbed exactly like attacks, 19 detectors, including GuardShield, lose the ability to separate the two in most cells, and attack-only evaluation mistakes over-blocking for robustness. The failure follows from what each tokenizer does to the perturbed text, and fixing the input with a script-aware canonicalizer helps more, and more reliably across data sources, than retraining on perturbed examples. These findings are written up as a research paper, and they define the next step for the project: a canonicalizing, multilingual GuardShield evaluated with paired benign controls.

CHAPTER 11
INDIVIDUAL CONTRIBUTION

[To be completed and confirmed by each team member. The entries below follow the roles recorded in the weekly progress reports.]

Shourya Solanki (A2305223569)

Lead architect. Implemented the Stage 1 classifier, the Stage 2 verifier, the streaming entropy monitor and the FastAPI proxy; trained and evaluated the models; ran the research audit on Kaggle and the mitigation study; prepared the research paper.

Rachit Ryan Chug (A2305223166)

Research analyst. Carried out the literature survey and the formulation of NLI-based fact verification; prepared the related-work positioning and the progress presentations.

Dhruv Raj Singh (A2305223191)

Data engineer. Curated and preprocessed the multi-source datasets, built the train, validation, test and out-of-distribution splits, and documented dataset provenance and licences.

REFERENCES

Chester, A. (2025). Tokenization confusion. XPN InfoSec Blog. https://blog.xpnsec.com/tokenization-confusion/

Conover, M., Hayes, M., Mathur, A., Xie, J., Wan, J., Shah, S., Ghodsi, A., Wendell, P., Zaharia, M. and Xin, R. (2023). Free Dolly: Introducing the world's first truly open instruction-tuned LLM. Databricks blog.

deepset (2023). prompt-injections [Dataset]. Hugging Face. https://huggingface.co/datasets/deepset/prompt-injections

Fairoze, J., Garg, S., Lee, K. and Wang, M. (2026). Bypassing prompt guards in production with controlled-release prompting. In USENIX Security Symposium. arXiv:2510.01529.

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T. and Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security.

Hackett, W., Birch, L., Trawicki, S., Suri, N. and Garraghan, P. (2025). Bypassing LLM guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems. In Proceedings of the First Workshop on LLM Security (LLMSEC). arXiv:2504.11168.

Han, S., Rao, K., Ettinger, A., Jiang, L., Lin, B. Y., Lambert, N., Choi, Y. and Dziri, N. (2024). WildGuard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of LLMs. In Advances in Neural Information Processing Systems (Datasets and Benchmarks Track).

He, P., Gao, J. and Chen, W. (2023). DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In International Conference on Learning Representations.

Inan, H., Upasani, K., Chi, J., Rungta, R., Iyer, K., Mao, Y., Tontchev, M., Hu, Q., Fuller, B., Testuggine, D. and Khabsa, M. (2023). Llama Guard: LLM-based input-output safeguard for human-AI conversations. arXiv:2312.06674.

Kuhn, L., Gal, Y. and Farquhar, S. (2023). Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In International Conference on Learning Representations. arXiv:2302.09664.

Lakera AI (2023). gandalf_ignore_instructions [Dataset]. Hugging Face. https://huggingface.co/datasets/Lakera/gandalf_ignore_instructions

Li, H., Liu, X., Zhang, N. and Xiao, C. (2025). PIGuard: Prompt injection guardrail via mitigating overdefense for free. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics, pp. 30420–30437.

Li, J., Cheng, X., Zhao, W. X., Nie, J.-Y. and Wen, J.-R. (2023). HaluEval: A large-scale hallucination evaluation benchmark for large language models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. arXiv:2305.11747.

Lin, S., Hilton, J. and Evans, O. (2022). TruthfulQA: Measuring how models mimic human falsehoods. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics. arXiv:2109.07958.

Liu, Z. (2026). Passing the test you trained on: Re-evaluating prompt-injection detectors for LLM agents. arXiv:2610.03448.

Manakul, P., Liusie, A. and Gales, M. J. F. (2023). SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. arXiv:2303.08896.

Meta (2025). Llama Prompt Guard 2 model card. https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-86M

Oblakov, N., Sadiekh, S. and Kokuykin, E. (2026). HiveTraceGuard-Pro: A compact generative guardrail for prompt injection, jailbreaks, and adversarial obfuscation. arXiv:2609.01046.

Perez, F. and Ribeiro, I. (2022). Ignore previous prompt: Attack techniques for language models. arXiv:2211.09527.

Protect AI (2024). LLM Guard: InvisibleText scanner. https://github.com/protectai/llm-guard

Sharma, R. K., Gupta, V. and Grossman, D. (2024). SPML: A DSL for defending language models against prompt attacks. arXiv:2402.11755.

Shire, Y. K. and Kim, S.-C. (2026). PIDS-Bench: Evaluating prompt-injection detectors under over-defense, obfuscation, and distribution shift. IEEE Access, 14, pp. 134184–134205.

Singh, S., Vargus, F., Dsouza, D., Karlsson, B. F. et al. (2024). Aya Dataset: An open-access collection for multilingual instruction tuning. arXiv:2402.06619.

Thorne, J., Vlachos, A., Christodoulopoulos, C. and Mittal, A. (2018). FEVER: A large-scale dataset for fact extraction and VERification. In Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics. arXiv:1803.05355.

Zhang, H., Xu, H., Luo, X., Liu, H., Chen, Y., Xiao, Z., Feng, Y., Guan, X., Zandsalimy, M. and Sushmita, S. (2026). Refusing everything looks safe: Restoring the benign arm to encoded-prompt evaluation. arXiv:2609.26176.

Zhao, H., Yuan, C., Huang, F., Hu, X., Zhang, Y., Yang, A. et al. (2025). Qwen3Guard technical report. arXiv:2510.14276.

APPENDIX A. RESEARCH PAPER

The research outcome of the project is the paper “Guards That Reject the Unfamiliar: Format-Triggered Over-Defense in Prompt-Injection Detectors” by Shourya Solanki, Rachit Ryan Chug, Dhruv Raj Singh and Abhishek Kaushal, prepared in the IEEE conference format (file paper1/paper/main.tex in the project repository, Appendix C). It reports Study 1, the tokenizer analysis and Study 2 of this report. [Update with the venue and the submission or publication evidence once available, as required by the NTCC guidelines.]

APPENDIX B. SECOND FAILURE MAP (PIDS-BENCH)

Figure B.1 repeats the Study 1 audit of Section 6.5 on 200 attack–benign pairs from PIDS-Bench, a second source with different attack styles. It uses the same colours and hatching as Figure 6.4.

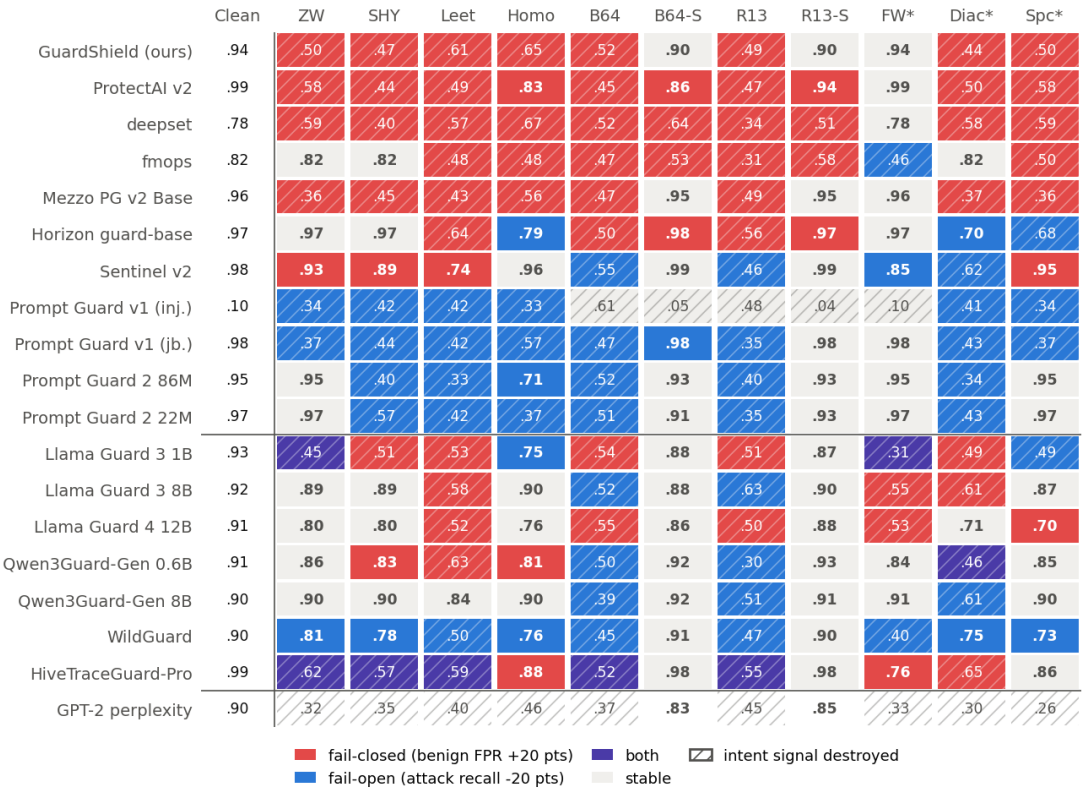


Figure B.1: Study 1 repeated on 200 PIDS-Bench pairs; encoding as in the main failure map

APPENDIX C. REPRODUCING THE RESULTS

All code, configurations and result files are in the project repository, https://github.com/Shourya3113/GuardShield-AI. The main commands are:

- python src/train_prescan_v3.py and python scripts/train_stage1_hard_negatives.py: train Stage 1.
- python src/evaluate_full_suite_v3.py: evaluate Stage 1 and Stage 2 and write docs/01 Reports/raw_eval_*.json.
- python src/proxy_server.py: run the sidecar proxy on port 8000.
- python -m guard_audit.run_audit build | score | metrics | report: the Study 1 audit harness.
- python paper1/study_c/run_study_c.py train | guards | score | metrics: the Study 2 pipeline.
- python paper1/paper/make_assets.py: regenerate every number, table and figure of the paper.
- python -m pytest tests/: the automated test suite (40 tests).

APPENDIX D. WEEKLY PROGRESS REPORTS

Weekly progress reports for weeks 1 to 12 are kept in the repository folder docs/02 Weekly WPRs and were submitted on Amizone as required. [Attach the signed copies and the minor project diary when binding.]